

EE186 Lab 4: Bit by Bit, Hertz by Hertz

Due: November 3, 2026

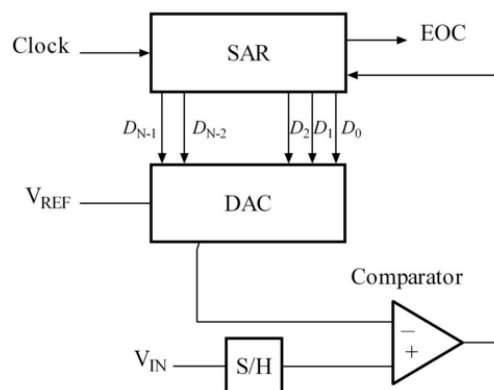
1 Introduction

Modern embedded systems frequently interact with the physical world, requiring the ability to convert analog signals into digital data and vice versa. In this lab, you will explore Analog-to-Digital Conversion (ADC) and Digital-to-Analog Conversion (DAC) using the Nucleo-L4R5ZI-P board. ADC is essential for reading real-world sensor data, while DAC allows microcontrollers to generate analog outputs, such as voltages, waveforms, or audio signals.

The objective of this lab is to convert an analog signal (light intensity) into a digital representation using the ADC, generate an analog output (sound) from a digital signal using the DAC, and use a photoresistor to measure light and control sound frequency. You will also explore sampling, quantization errors, and the importance of accurate signal conversion. By the end, you will have a fully functional light-controlled theremin, an electronic musical instrument played without physical contact.

2 ADC

Analog-to-Digital Converters (ADCs) convert analog signals into digital signals that can be processed by the microcontroller. The STM32L4 microcontroller uses a type of ADC called the Successive Approximation Register (SAR). The SAR ADC requires a DAC to function. It works by performing a binary search to find the DAC signal that best approximates the signal. That is, by successively turning on DAC bits one at a time, the ADC uses a comparator to see whether the DAC output is higher or lower than the input signal. Then it adjusts the DAC output again repetitively to get a better and better approximation, until it has no more solution. See this [.gif](#) or the figure below.



Higher output bit resolutions require more approximations and therefore more time. This limits the sampling rate of ADC. Since the ADC returns the DAC value that best represents the input voltage signal, the binary

ADC data register value D for an n -bit ADC would be set in relation to the reference voltage:

$$V_{in} \approx \frac{D}{2^n} V_{ref}$$

Configure the onboard ADC and connect it with a potentiometer. Test the ADC by printing the digital output on a terminal while changing the values of the potentiometer. You should expect to see a reading of 0 when the potentiometer is turned fully in one direction, and then a non-zero maximum value when turned in the other direction.

Next, replace the potentiometer with the provided photodiode. Using the photodiode, we can determine the light intensity in lux by measuring the voltage drop in a voltage divider. Construct a voltage divider circuit with a $10\text{ k}\Omega$ resistor and a photodiode to build a light meter. The resistance value of $10\text{ k}\Omega$ gives good sensitivity for typical room conditions. Using a single light source (e.g. desk lamp in a dark room), make several ADC measurements over a light intensity range of about 1000 lux. Use a light meter app on your smartphone for the light measurements.

Deliverables

- (1 Point) What are the different bit resolutions can you set the ADC? What is the maximum sampling rate at each of these resolutions? Where did you find this information in the datasheet?
- (1 Point) Why do you need a voltage divider circuit? What would happen if the photodiode is directly connected to the board?
- (2 Point) Submit circuit diagrams for both configurations used in this part of the lab. The first should illustrate the potentiometer connected as a variable voltage divider, and the second should show the voltage divider circuit using a photodiode and a $10\text{ k}\Omega$ resistor. Both diagrams should clearly indicate the supply voltage V_{in} , the output voltage V_{out} , and the ground reference.
- (3 Point) Submit a video demonstrating how the output of the ADC changes in the terminal when the light changes. Submit the code.

3 DAC

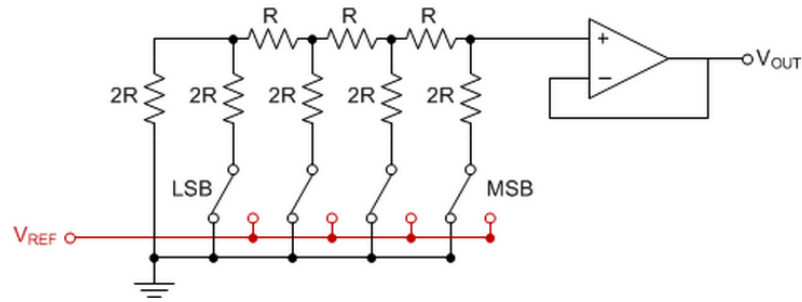
The DAC allows the microcontroller to generate analog signals from digital values. In this section, you will configure the DAC to output a varying voltage that can be visualized on a logic analyzer or measured with a multimeter.

The DAC on the STM32L5 microcontroller is a very common type of DAC called an R-2R Ladder, as shown in the figure below. In this circuit, the microcontroller controls the switches that connect V_{ref} to a resistor ladder. This ladder forms a series of voltage dividers that each divide V_{ref} by some power of 2. When multiple switches are activated, the voltages sum together on the output. By setting different bits in the DAC output register, you set the output voltage.

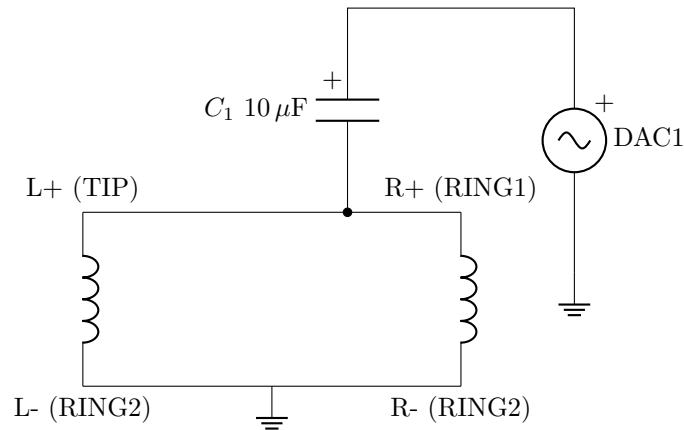
For an n -bit DAC with the binary input $B = B_n B_{n-1} \cdots B_1 B_0$,

$$\begin{aligned} V_{out} &= \frac{V_{ref}}{2} B_n + \frac{V_{ref}}{4} B_{n-1} + \cdots + \frac{V_{ref}}{2^n} B_0 \\ &= \frac{B}{2^n} V_{ref} \end{aligned}$$

First, configure both the ADC and the DAC. Test the DAC output by reading it with the ADC. Increment the DAC data value until it reaches the maximum and then decrement it to the minimum value and repeat. Connect the DAC output to the ADC input with a jumper wire. Print the DAC, ADC, and their difference (ADC - DAC) as voltage values on a serial terminal.



Once DAC is working correctly, configure the STM32L5's DAC to output a sine waveform. Write values to the DAC output register at a controlled rate. Use a lookup table to generate a sine wave. Then, connect the DAC output to a TRRS audio breakout board (see the figure below for wiring), which will allow you to output the generated signal to earphones. Observe the signal through a logic analyzer and listen to how different waveforms sound through the earphones. Experiment with different frequencies to hear the difference between low and high-frequency waveforms.



You will find [this](#) ST manual useful when generating a sine wave.

Deliverables

- (1 Point) If $B = 9$, what is V_{out} ? Express your answer in terms of V_{ref} . Use the DAC mode that provides the best resolution available on your MCU (refer to the data sheet).
- (3 Point) Submit a screenshot of your serial terminal printing DAC, ADC, and their difference as voltage values as you increment and decrement the DAC output.
- (3 Point) How does the sampling rate of the DAC affect the quality of the output waveform? What are the differences in sound perception between sine, square, and sawtooth waves? Why do we need a capacitor in the signal path when sending an audio signal to a speaker or earphones?
- (5 Point) Submit your code generating the middle C pitch (262 Hz). Submit a screenshot of that sine wave on a logic analyzer software.

4 Musical Instrument

Let's bring everything together and build a [theremin](#). A theremin is an electronic musical instrument played without physical contact.

In this section of the lab, we will make our own version of this, except, instead of using electromagnetic fields to detect the distance between your hand and the instrument, we will use an LDR. As your hand moves closer to the photodiode, less light shines on it. We can use this phenomenon to get a rough estimate of the distance between the user's hand and the instrument. Then we will output an audio frequency that is dependent on the distance between the hand and the instrument. Use the TRRS audio breakout board from the last section to play the generated music.

In the previous part of the lab, you implemented a lookup table containing values for a sinusoidal waveform. Modify this code to define a new function `changeNote(frequency)`. This function should generate a sine wave at the given frequency. Connect your earphones and verify that you can hear a simple musical scale. You can also confirm the waveform output using a logic analyzer.

Next, modify the project to act as a Theremin by using the photoresistor as the input instead of manually specifying the frequency. Update your code so that the argument to `changeNote()` varies based on the light intensity detected by the LDR. Choose an appropriate frequency range (high and low notes) that produces an interesting and audible range of sounds as your hand moves over the sensor.

Deliverables

- (5 Point) Submit a short video demonstration of your working Theremin along with your source code.
- (Optional, +2 Point) Extra credit for those who produce musical or harmonic sounds!